PhD Evaluation Task Report

Reproduction and Analysis of "Addressing Distributional Shifts in Operations Management: The Case of Order Fulfillment in Customized Production"

Paper title	Addressing distributional shifts in operations management: The case of order fulfillment in customized production
Authors	Julian Senoner, Bernhard Kratzwald, Milan Kuzmanovic, Torbjørn Netland (ETH Zurich) and Stefan Feuerriegel (LMU Munich)
Publication	*Production and Operations Management*, Vol. 32, No. 10, October 2023 (published online April 2023)
Original code and data	github.com/mkuzma96/CustomProd
My GitHub repository	github.com/rotoncsedu/phd_evaluation_task
Google Drive folder	Project files (Google Drive)

Submitted by

Md. Al Imran

Rangpur, Bangladesh

Summary

This report documents my work on the assigned PhD evaluation task: selecting a suitable research paper with public code and data, running and understanding the code, extending it with additional experiments and visualizations, and comparing my results with those published by the authors. The selected paper studies how manufacturers can obtain reliable throughput time predictions, and therefore reliable production schedules, when future orders differ systematically from historical ones (a distributional shift). The authors propose training a neural network with Wasserstein distance-guided representation learning (WDGRL), which uses the unlabeled features of the forthcoming orders to align the learned representations of old and new orders.

My replication reproduces the central finding qualitatively. As the simulated shift grows, the prediction error and scheduling cost of the Elastic Net and neural network baselines increase markedly, whereas WDGRL remains comparatively stable. The numerical values differ moderately from those in the paper, and the report discusses likely reasons and the limits of what a single replication can establish.

Part I: Workflow and AI Use Declaration

AI Use Declaration

The table below records how AI tools supported each stage of the task and what I did to verify or modify their output. All AI-generated material was reviewed before use, and I take responsibility for the content of this report.

Table A: AI tools used and verification performed

Task	Tool(s)	What I asked	What the AI produced	What I changed or verified
Search for candidate papers	ChatGPT, Claude	Find and summarize at least five best-matched articles, including public data and code availability.	Each tool returned five papers with short summaries.	Examined the suggestions and shortlisted three candidate papers.
Understanding the paper and source code	ChatGPT, Claude	Clarify specific equations and code segments.	Explanations of the requested equations and code.	Read the explanations critically and consulted other resources where required.
Code rewriting and visualization	Claude, Gemini	Add improved visualizations and Markdown documentation.	Plotting code and Markdown text.	Examined and modified the output where required, then inserted it into my `test.ipynb` and `test.py` scripts.
Result analysis	ChatGPT	Clarify the results and the possible reasons for them.	Explanations of the results.	Read and understood the explanations, then ran the code and analyzed the results myself.
Report writing	ChatGPT	After writing the full report myself, asked for error correction, formatting, and polishing for a more formal style.	A revised report file.	Checked the revised report and made corrections.
Presentation preparation	Gemma	Request an outline for a 10-minute presentation.	A presentation outline.	Used the outline as guidance for slide arrangement and then created the slides myself.

Task Workflow

Task 1: Find a suitable paper (target completion: Day 1, 15 October 2026)

Step 1: Search for candidate papers

1. I browsed *Decision Support Systems* on ScienceDirect for recent related articles and visited two to three other journal sites, limiting manual searching to avoid spending excessive time.
2. I gave ChatGPT and Claude a description of my research interests and previous experience, and asked each to find and summarize at least five best-matched articles, including the availability of public data and code.
3. I briefly studied the suggested articles, verified them through independent internet searches, and created a shortlist of three candidate articles.
4. I gave the shortlist to both tools again and asked for details of each paper, the authors' repository link, and the paper download link.

Step 2: Scoring the candidate papers

1. Downloaded the original articles and cloned the repositories.
2. Read the papers carefully to understand the technical details.
3. Examined the data, code, README file, and other documentation.
4. Assessed the code complexity, the algorithms used, and the data representation.

Step 3: Trial run and final selection

This final step decided the selection, because a paper is only useful if its code can actually be run (for example, the code of Candidate 1 required a GPU).

1. Installed the required packages to set up a runnable environment.
2. Checked for any license requirements for running the code.
3. After a clean environment setup on my own PC, made a test run to confirm that everything worked.

Based on this process, I selected the paper "*Addressing distributional shifts in operations management: The case of order fulfillment in customized production*," published in *Production and Operations Management* (Vol. 32, No. 10, October 2023).

Task 2: Run and understand the full code (target completion: Day 2, 16 October 2026)

1. Ran the code locally on a Lenovo IdeaPad Pro 5 16IRH8.
2. Studied each part of the code, using ChatGPT and Claude to clarify any segment that was unclear or unfamiliar.
3. Compared the implementation with the methodology described in the authors' paper.
4. Examined the script dependencies, model structure, and hyperparameters.
5. Analyzed the outputs of `prediction.py` and `schedule.py` and stored them in appropriately named directories.
6. Compared my results with those reported in the paper.

Task 3: Rebuild the code, test different parameters, and add visualizations (target completion: Day 3, 17 October 2026)

1. Created a local `test.py` file and ran the code segment by segment rather than as a full script, which made debugging considerably easier.
2. Tested the output under different parameter settings to understand how each parameter affects the results.

3. Created a `.ipynb` version of the code for interactive, block-by-block execution, which also helped me understand the relationship between the model configuration and the output.
4. Wrote additional code for visualizations: training and test data distributions, feature correlations, intermediate outputs, predictions of the different models, and a visual comparison of the final results. Claude and Gemini assisted with improving these visualizations.

Task 4: Result comparison, report, presentation, and file organization (target completion: Day 4, 18 October 2026)

1. Compared my results with the authors' results.
2. Analyzed the discrepancies, with ChatGPT used to help clarify possible causes.
3. Wrote a formal report myself and then used an AI tool to correct errors and polish the style.
4. Prepared a 10-minute presentation, using Gemma to obtain an outline and then building the final slides myself.
5. Organized all files with proper names, made the repository public, and shared the links.

Part II: Technical Report

1. The Business Problem

Manufacturers aim to deliver orders on time. Finishing late harms customers, whereas finishing early creates storage costs and exposes the manufacturer to the risk of rework if the customer changes the design. To avoid both outcomes, factories first predict how long each order will take (the **throughput time**) and then schedule the orders around these predictions. This two-step workflow is known as **predict-then-optimize**.

The difficulty arises in customized production, where each customer order has different specifications. Past data then look different from future data, a phenomenon known as a **distributional shift**. Most machine learning methods assume that training data and future data come from the same distribution, so their predictions deteriorate when this assumption fails, and worse predictions lead to worse schedules. The central question of the paper is therefore how a manufacturer can obtain reliable throughput time predictions, and hence reliable schedules, for a forthcoming production setting whose outcomes are not yet known.

The problem is a case of **covariate shift**: the distribution of the order features changes between the historical and the forthcoming setting, while the relationship between features and throughput time is assumed to remain stable. This assumption is important for interpreting the method and its limitations (Section 9).

2. The Case Company and Data

The authors worked with Aker Solutions, a Norwegian engineering company. Its job shop produces large metal pipe pieces, called spools, for oil platforms, and each spool constitutes one production order. Two production settings are compared:

- **Setting A (historical):** components for the Johan Castberg floating production vessel, comprising 5,830 spools produced between January and September 2019.
- **Setting B (forthcoming):** components for the Johan Sverdrup riser platform modification, comprising 3,866 spools produced between September 2019 and April 2020.

The two periods do not overlap. Each spool is described by 20 features, such as its length, weight, number and length of welds, insulation thickness, pressure and temperature specifications, and planned job length (Figure 1). Throughput times are known for setting A. For setting B, the company has the features but not the outcomes at scheduling time, because these lie in the future. In domain adaptation terminology, A is the labeled source domain and B is the unlabeled target domain.

Feature	Description	Feature	Description
x_1	Material multiplication factor	x_{11}	Demolition multiplication factor
x_2	Insulation thickness around the spool	x_{12}	Min. design temperature of the fluid/gas in the spool
x_3	Length of the spool	x_{13}	Test pressure of the fluid/gas in the spool during testing
x_4	Dry weight of the spool	x_{14}	Operational pressure of the gas/fluid in the spool
x_5	Summed length of all welds of the spool	x_{15}	Max. operational temperature of the fluid/gas in the spool
x_6	Number of welds	x_{16}	Max. design temperature of the fluid/gas in the spool
x_7	Average weld diameter	x_{17}	Number of different materials needed to produce the spool
x_8	Average weld thickness	x_{18}	Number of different tasks to produce the spool
x_9	Planned length of the job	x_{19}	Max. average historical delay of the material deliveries for the spool
x_{10}	Planned sum of workline hours	x_{20}	Revision multiplication factor

Figure 1: Description of the 20 order features (x_1–x_{20})

3. Data Analysis

Table 1 summarizes the two settings as reported in the paper.

Table 1: Overview of the data

Property	Setting A	Setting B
Production orders	5,830	3,866
Time period	Jan–Sep 2019	Sep 2019–Apr 2020
Mean actual throughput time	32.0 days	35.2 days
Features per order	20	20
Throughput labels available when predicting B?	Yes	No

In the research experiment, the actual labels of B are retained for evaluation only, and the models are not allowed to use them for training. In practice, these future throughput times are precisely what the production manager does not yet know.

Semi-synthetic data generation

For reasons of data privacy, the authors work with semi-synthetic data. They first derive summary statistics of the real product features and store them in the file `rw_data.npz`, which contains the following arrays:

- `due_dates`: the due dates of the orders;
- `mean_train` and `cov_train`: the mean vector and covariance matrix of the features of setting A;
- `mean_test` and `cov_test`: the mean vector and covariance matrix of the features of setting B;
- `sig_y`: the scale of the noise used to generate synthetic outcomes.

Features for setting A are then sampled from a multivariate normal (Gaussian) distribution whose mean is `mean_train` (the average of each input feature) and whose covariance is `cov_train` (how the features vary and relate to each other). Features for setting B are sampled in the same way, except that the mean is moved from A toward B by an amount controlled by the shift parameter θ; a larger θ means a stronger distribution shift. Figure 2 visualizes the shift for $\theta = 4$ and shows clearly how the feature distribution of setting B departs from that of setting A.

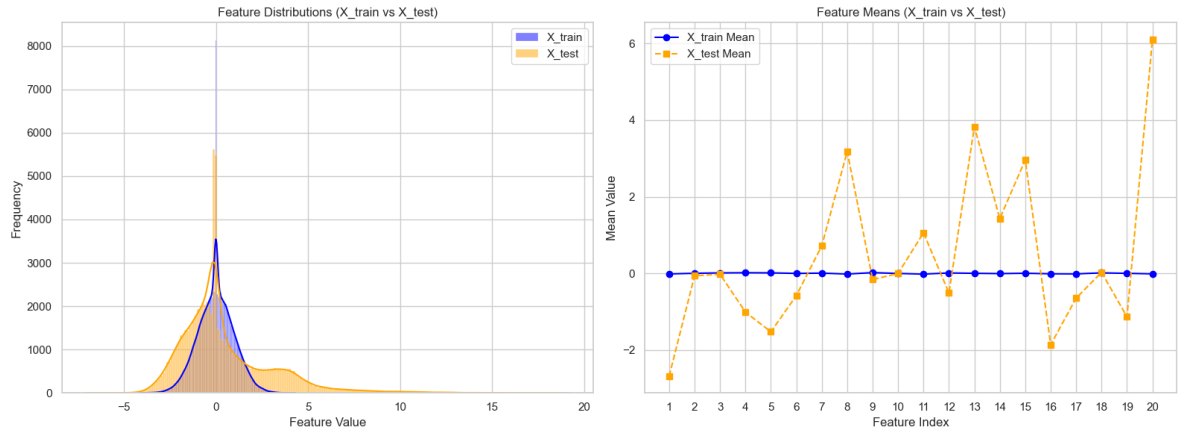


Figure 2: Distribution shift between setting A (X_train) and setting B (X_test) for θ = 4. Left: distributions of the feature values. Right: mean of each of the 20 features

Although the feature means change, the correlation structure among the features remains visibly similar in the two settings (Figure 3). The shift is therefore mainly a shift in location rather than a change in how the features relate to one another.

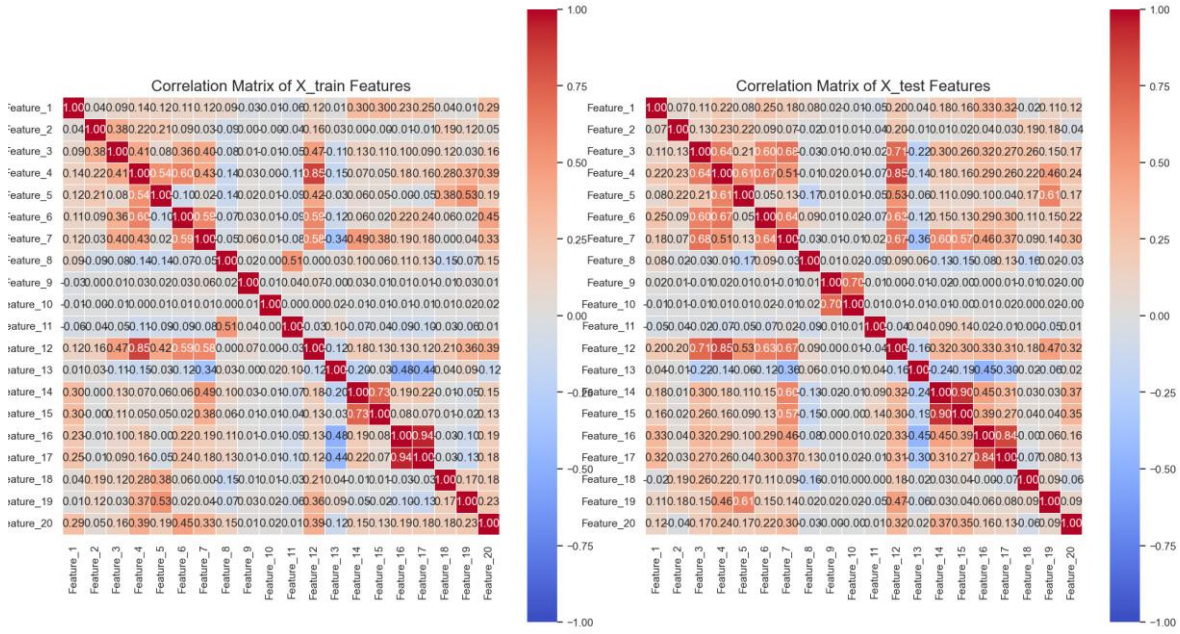


Figure 3: Correlation matrices of the features of setting A (left) and setting B (right)

Throughput times are generated as $y = \varphi(x) + \varepsilon$, where φ is a random forest fitted to real Aker data (stored in `rf_model.pickle`) and ε is random noise whose scale is given by `sig_y`. The resulting synthetic targets are used to train the models on setting A and to evaluate them on setting B. Figure 4 compares the target distributions of the two settings: the test targets are slightly displaced and more concentrated than the training targets, which is the consequence of the feature shift passing through φ. Note also that, because the noise is Gaussian, some synthetic throughput times are negative, which is not physically meaningful and is a by-product of the semi-synthetic design.

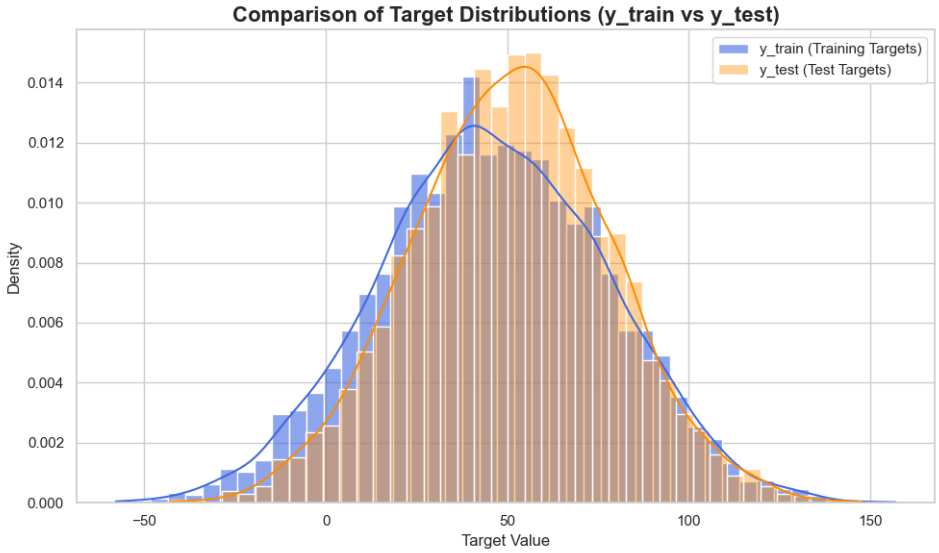


Figure 4: Distribution of the synthetic target values for the training data (setting A) and the test data (setting B)

4. The Two-Stage Decision Problem

Stage 1: Prediction. A model is trained on setting A to predict the throughput times of the orders in setting B. The loss is the mean absolute error (MAE) over the m orders of setting B, where y_i^B is the actual and $\hat{y}_i^B$ the predicted throughput time of order i. The authors chose this loss so that prediction errors and scheduling costs are both measured in the same absolute-value (L1) manner.

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m} \left| y_i^B - \hat{y}_i^B \right|$$

Stage 2: Scheduling. Each order i has a due date d_i, and finishing early or late costs money per time unit. If order i starts at time t and has throughput time $\hat{y}_i$, its completion time is $t + \hat{y}_i$, and the cost is

$$C_{it} = c_{\text{early}} \max\{0,\ d_i - (t + \hat{y}_i)\} + c_{\text{tardy}} \max\{0,\ (t + \hat{y}_i) - d_i\}$$

The first term penalizes finishing before the due date, and the second penalizes finishing after it. The scheduler uses the *predicted* times $\hat{y}_i$, whereas the final cost is evaluated with the *actual* times y_i. Poor predictions therefore translate directly into poor schedules. The decision variable is

$$z_{it} = \begin{cases} 1, & \text{if order } i \text{ starts at time } t, \\ 0, & \text{otherwise.} \end{cases}$$

and the scheduling problem is

$$\min_{z_{it} \in \{0,1\}} \sum_{t=1}^{T} \sum_{i=1}^{m} z_{it}\ C_{it}$$

subject to

$$\psi(z, \hat{y}, t) \le K_t \qquad \forall\, t,$$

$$\sum_{t=1}^{T} z_{it} = 1 \qquad \forall\, i.$$

Here, ψ counts the number of orders being processed in parallel at time t, and K_t is the available production capacity at that time.

This is an **integer linear program** that selects the start time of each order so as to minimize the total cost, subject to two rules: no more than K_t orders may run in parallel at any time, and every order is scheduled exactly once. It is solved with the GLPK solver (branch-and-cut). For computational reasons, the problem is limited to the 100 orders with the earliest due dates.

5. The Proposed Method

Core idea: learn a representation that looks the same for A and B. The authors use Wasserstein distance-guided representation learning (WDGRL), which trains three small neural networks jointly:

1. **Feature extractor** f_e: maps the 20 raw features into a shared latent space. Input: the 20 order features from either A or B. Output: a latent representation $h = f_e(x)$.
2. **Regressor** f_r: predicts the throughput time from the latent representation. Input: h from setting A. Output: the predicted throughput time $\hat{y} = f_r(h)$.
3. **Critic** f_c: estimates how different the latent distributions of A and B are (the Wasserstein distance). Input: representations from A and B. Output: a real-valued score rather than a probability.

Taken together, the prediction pipeline is

$$h = f_e(x), \qquad \hat{y} = f_r(h) = f_r\big(f_e(x)\big)$$

The critic is used only during training to guide representation learning; it is not needed to predict throughput times after training. Table 2 summarizes the objective of each component. The critic and the feature extractor have opposing objectives, whereas the regressor cooperates with the feature extractor on prediction.

Table 2: Components of WDGRL and their objectives

Component	Objective	Intuition
Critic f_c	Maximize the Wasserstein loss	Detect the distributional differences between A and B
Feature extractor f_e	Minimize the regression loss and the estimated distance	Preserve predictive information while reducing domain differences
Regressor f_r	Minimize the regression loss	Predict the source throughput times accurately

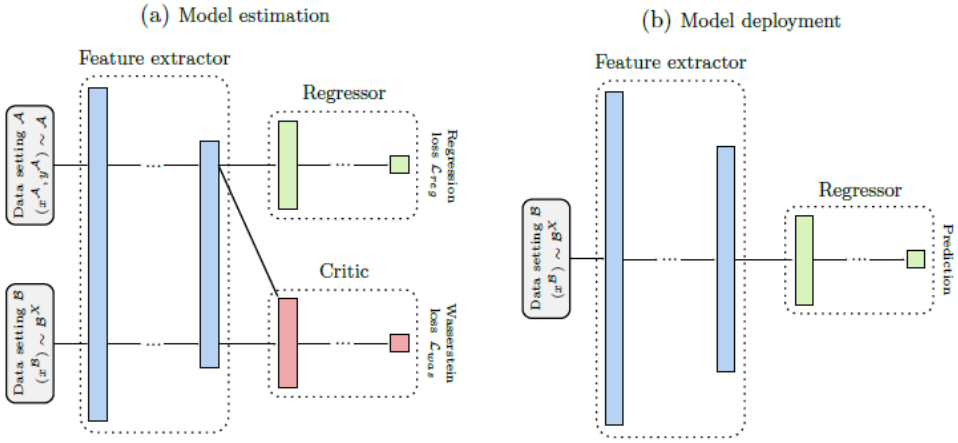


Figure 5: Model architecture of WDGRL

Wasserstein distance

The Wasserstein-1 distance, also called the Earth Mover's Distance, measures the minimum cost of transporting probability mass so as to transform one distribution into another. Formally,

$$W_1(P_A, P_B) = \inf_{\gamma \in \Pi(P_A, P_B)} \mathbb{E}_{(u,v)\sim\gamma} \parallel u - v \parallel$$

where P_A and P_B are the two probability distributions, γ is a joint distribution (a coupling) of observations from the two distributions, $\Pi(P_A, P_B)$ is the set of all couplings with the correct marginal distributions, and $\|u - v\|$ is the cost of transporting mass between two points. The infimum selects the least costly valid transport plan.

Computing the infimum directly is expensive. In practice, WDGRL relies on the Kantorovich–Rubinstein dual form, in which the critic f_c is a 1-Lipschitz function and the distance is estimated by the difference of its mean values on the two domains:

$$W_1(P_A, P_B) = \sup_{\|f_c\|_L \leq 1} \mathbb{E}_{h\sim P_A}[f_c(h)] - \mathbb{E}_{h\sim P_B}[f_c(h)]$$

This explains why the critic outputs an unbounded real-valued score: the larger the gap between its average scores on A and B, the larger the estimated distance. Figure 6 shows the univariate

Wasserstein distance of each raw feature between settings A and B; the distances differ strongly across features, so the shift is concentrated in a subset of the features rather than being spread evenly.

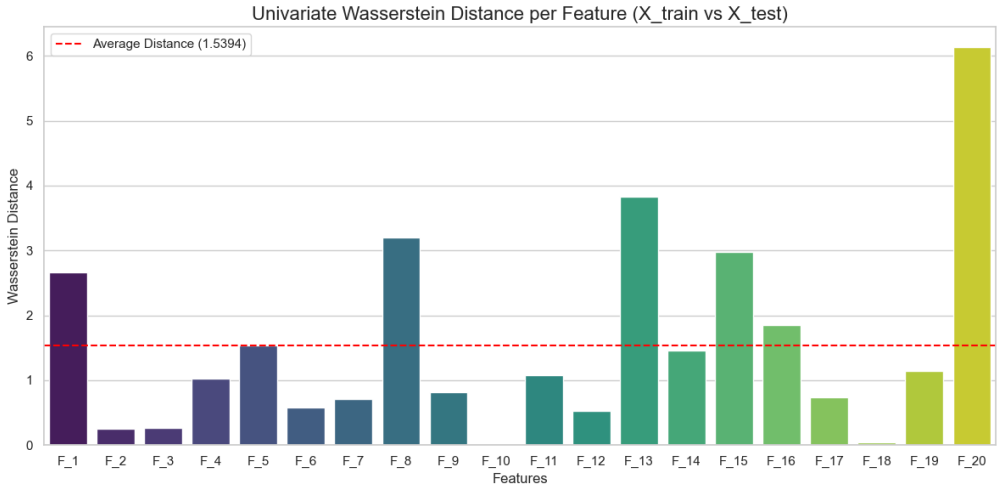


Figure 6: Univariate Wasserstein distance for each feature of settings A and B

Training objective

The method pursues two competing goals in a minimax formulation:

- **Objective 1:** predict the throughput times of A as accurately as possible (uses the labels of A).
- **Objective 2:** make the latent representations of A and B as similar as possible (uses only the *features* of B, not its labels).

In its standard formulation (Shen et al., 2018), on which the method builds, the objective can be written as

$$\min_{f_e, f_r} \max_{f_c} \; \mathcal{L}_{\text{reg}}(f_e, f_r) + \lambda \, \mathcal{L}_{\text{wd}}(f_e, f_c)$$

where L_{reg} is the regression loss on setting A, L_{wd} is the Wasserstein distance estimate between the latent representations of A and B, and λ weights the alignment term. The exact loss functions and weights follow the authors' code. The key insight of the paper can be summarized as the combination

$$\text{Source prediction loss} \; + \; \text{Target representation alignment}$$

The critic estimates distributional differences in the latent space, the feature extractor attempts to reduce those differences, and the regressor ensures that the representation remains useful for prediction.

6. The Experiments

Because the true completion times of setting B were treated as unknown, the main experiments are **semi-synthetic**:

- Compute the mean and covariance of the real Aker features for A and B.
- Sample synthetic features for A from a Gaussian distribution built on the statistics of A. For B, shift the mean from A toward B by a factor $\theta \in \{1, 2, 3, 4\}$; larger θ means a stronger shift.
- Generate throughput times as $y = \varphi(x)$ + noise, where φ is a **random forest** fitted to real Aker data. A tree-based generating function was chosen deliberately so that neither the linear nor the neural network models enjoy an unfair advantage.

- Use capacity K_t = 70, equal early and late costs, and 10 repeated runs (mean ± standard deviation reported).

Three prediction models are compared:

1. **Elastic Net**, a regularized linear regression model;
2. **Baseline neural network**, a non-linear model;
3. **WDGRL**, the proposed Wasserstein distance-guided representation learning model.

These models were selected to keep the comparison fair: one linear model (Elastic Net), one non-linear model (the neural network), and data generated by a random forest, which favors neither of them. Moreover, the authors use the same neural network as the baseline and as the backbone of WDGRL, without extra hidden layers and with the same hyperparameters, so that any improvement is attributable to the alignment mechanism and not to a larger model.

Hyperparameters are selected by five-fold cross-validation on the source data. The supplement reports a learning rate of 0.0005; feature-extractor sizes of 8, 16, 32, or 64 neurons; a regressor with 8 neurons; 50 or 100 epochs; batch sizes of 32 or 64; several kernel-regularization values; and dropout rates of 0.4, 0.5, or 0.6. All selected hyperparameters are saved in the data folder of the code.

Code implementation workflow

The authors' code follows the logic summarized in Table 3.

Table 3: Code workflow and its research purpose

Script operation	Research purpose
Run `cross_validation.py`	Generate the hyperparameters and save them in the data folder under appropriate file names
Load `rw_data.npz`	Obtain the estimated means, covariance matrices, and outcome-noise scale
Load `rf_model.pickle`	Obtain the fitted data-generating function
Sample training features from the distribution of A	Construct the source-domain training data
Predict with the random forest and add noise	Construct the simulated source labels
Sample target features with the θ-dependent mean	Construct target-domain inputs with a controlled shift
Generate target labels for evaluation	Provide simulated ground truth to measure performance
Train Elastic Net, neural network, and WDGRL	Compare the predictive approaches
Run the scheduling optimization	Evaluate the operational effect of the predictions

7. Result Analysis

For each value of θ, the authors report the mean over 10 independent simulation runs. I repeated the experiment with the authors' released code and report my own results in the same format, so that the two sets of results can be compared directly. In all tables below, the lowest value in each column is shown in bold (excluding the Oracle).

7.1 Prediction performance

Lower MAE is better. Table 4 gives the means reported in the paper and Table 5 gives the means from my replication.

Table 4: Prediction results (MAE) reported by the authors

Method	θ = 1	θ = 2	θ = 3	θ = 4
Elastic Net	24.3	25.3	28.1	31.5
Neural network	23.8	24.4	27.2	31.4
WDGRL	**23.5**	**23.7**	**24.0**	**23.8**

Table 5: Prediction results (MAE) from my replication

Method	θ = 1	θ = 2	θ = 3	θ = 4
Elastic Net	23.5	26.6	27.0	29.6
Neural network	**22.6**	25.5	25.5	28.4
WDGRL	23.2	**23.9**	**23.9**	**22.6**

The pattern matters more than any single number. In the paper, as the shift increases from $\theta = 1$ to $\theta = 4$, the MAE of the neural network rises from 23.8 to 31.4 and that of Elastic Net from 24.3 to 31.5, while the MAE of WDGRL stays within 23.5–24.0. In my replication, the MAE of the neural network rises from 22.6 to 28.4 and that of Elastic Net from 23.5 to 29.6, whereas WDGRL remains within 22.6–23.9. At the strongest shift ($\theta = 4$), WDGRL reduces the MAE of the neural network baseline by 24.2% in the paper and by 20.4% in my replication. At the mildest shift ($\theta = 1$), the differences between the three models are small, and in my replication the neural network is in fact slightly better than WDGRL (22.6 versus 23.2). WDGRL therefore pays off mainly when the shift is substantial, and it does not appear to cost much accuracy when the shift is mild.

This difference in behavior has a clear explanation. The ordinary neural network learns a predictor by minimizing the error on the source domain only. WDGRL additionally uses the unlabeled target features to shape the representation, which gives it a mechanism to adapt to the changed feature distribution without target labels. The results therefore support the claim that WDGRL is more robust to the particular shifts represented in this experimental setup.

Figures 7 and 8 illustrate this at $\theta = 4$. In Figure 7, the predictions of WDGRL (right panel) are located closer to the diagonal than those of the two baselines, whose predictions are systematically too high. Figure 8 shows the same effect in terms of distributions: the predictions of Elastic Net and the neural network are centered at roughly 68–75, well above the true targets, which are centered at about 50, whereas the WDGRL predictions are centered slightly below the true center. The WDGRL predictions are also narrower than the true targets, which is expected for any regression model trained on noisy outcomes, because it estimates the conditional mean and cannot reproduce the noise.

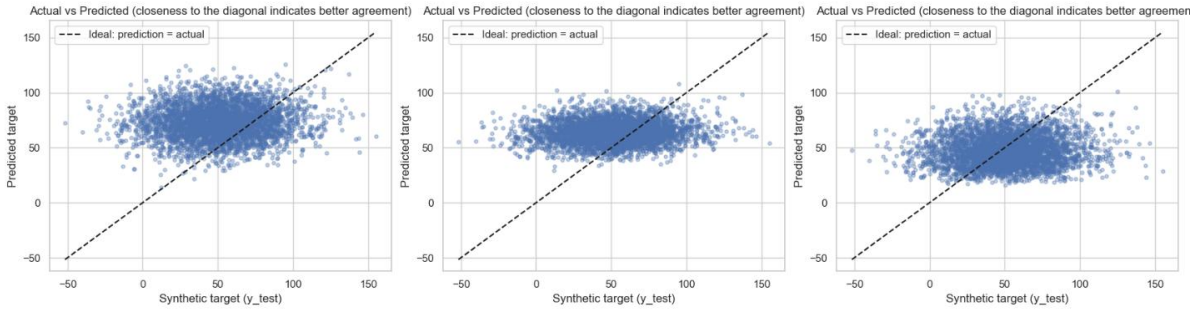


Figure 7: Actual versus predicted throughput times at θ = 4 (left to right: Elastic Net, neural network, WDGRL). Points closer to the dashed diagonal indicate better agreement

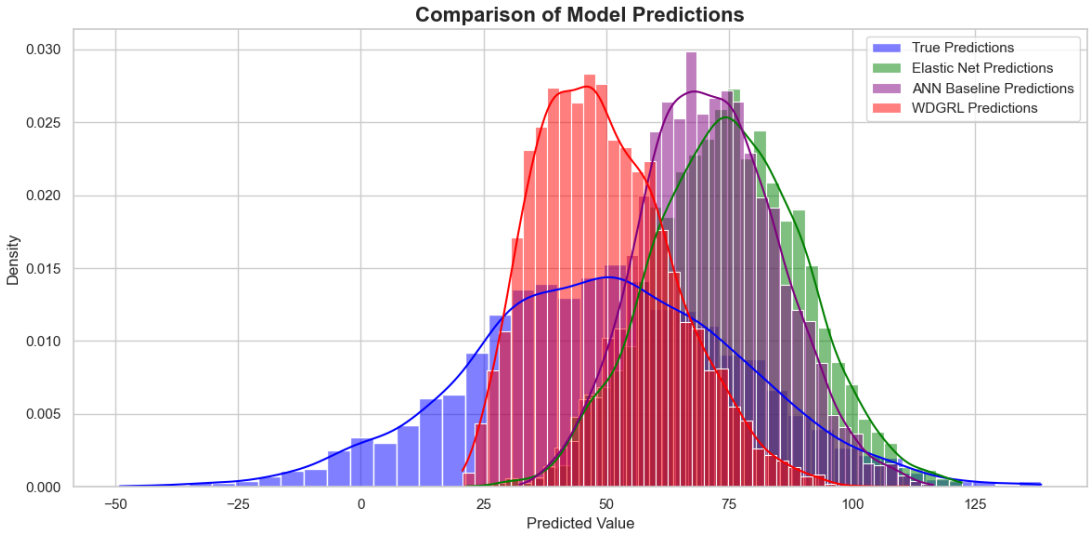


Figure 8: Comparison of the predictions of the three models with the true target values at θ = 4

7.2 Scheduling performance

The more operationally meaningful outcome is the realized scheduling cost, given in Table 6 (paper) and Table 7 (my replication). Lower is better.

Table 6: Scheduling cost reported by the authors

Method	θ = 1	θ = 2	θ = 3	θ = 4
Elastic Net	2,989.3	3,355.0	3,931.0	4,453.9
Neural network	2,933.1	3,209.4	3,870.9	4,568.4
WDGRL	**2,782.2**	**2,802.7**	**2,995.2**	**2,954.2**
Oracle	417.9	503.0	518.4	600.2

Table 7: Scheduling cost from my replication

Method	θ = 1	θ = 2	θ = 3	θ = 4
Elastic Net	2,940	3,439	3,916	4,205
Neural network	2,820	3,236	3,839	4,259
WDGRL	**2,753**	**2,811**	**3,033**	**2,668**
Oracle	468	423	634	575

The oracle is a lower-bound benchmark that has access to the true, noise-free outcome-generating function. It cannot be used in practice; its purpose is to show how much room remains between realistic and idealized predictions. Even the oracle incurs a positive cost, because the realized throughput times contain noise that no predictor can remove.

The scheduling costs confirm the prediction results. In the paper, from $\theta = 1$ to $\theta = 4$ the cost of the neural network increases by 55.8% and that of Elastic Net by 49.0%, whereas the cost of WDGRL increases by only 6.2%. In my replication, the corresponding increases are 51.0% and 43.0% for the baselines, while the cost of WDGRL decreases by 3.1% (from 2,753 to 2,668), a small decline that I attribute to random variation rather than a genuine improvement under stronger shift. At $\theta = 4$, the cost of WDGRL is roughly 4.9 times the oracle cost in the paper and 4.6 times in my replication, so a substantial gap to ideal predictions remains even for the best model.

Table 8 expresses the benefit of WDGRL as the relative reduction with respect to the neural network baseline. The benefit grows with the severity of the shift in both the paper and my replication: it is small at $\theta = 1$ (and slightly negative for the MAE in my replication) and exceeds one third of the scheduling cost at $\theta = 4$.

Table 8: Relative reduction achieved by WDGRL compared with the neural network baseline (%)

Metric and source	θ = 1	θ = 2	θ = 3	θ = 4
MAE, paper	1.3	2.9	11.8	24.2
MAE, my replication	-2.7	6.3	6.3	20.4
Scheduling cost, paper	5.1	12.7	22.6	35.3
Scheduling cost, my replication	2.4	13.1	21.0	37.4

7.3 Comparison between my results and the paper

My replication reproduces the ranking of the methods and the qualitative pattern at every shift level: Elastic Net and the neural network deteriorate as θ grows, whereas WDGRL remains stable. The numerical values differ moderately. Averaged over the four shift levels, the scheduling costs in my replication deviate from the published ones by 2.5% for Elastic Net, 3.1% for the neural network, and 3.1% for WDGRL. The largest absolute MAE difference is 3.0 (neural network at $\theta = 4$: 31.4 in the paper versus 28.4 in mine). The oracle costs show the same kind of variability: they increase monotonically with θ in the paper, whereas in my run the value at $\theta = 2$ (423) is lower than at $\theta = 1$ (468).

Several factors can plausibly explain these differences. Features and noise are sampled randomly, and only 10 repetitions are averaged, so the sampling variability of the mean remains non-negligible. Neural network training is stochastic (weight initialization, mini-batch order, dropout), and differences in random seeds, package versions, and hardware can change the results even when the code is identical. The scheduling problem is also sensitive to small changes in the predicted times, which can alter which orders compete for the same capacity. I did not isolate which of these factors dominates; doing so would require fixing seeds and repeating the experiment many times, which I identify as future work.

Given that the ranking and the qualitative behavior match, I conclude that my results support the central claim of the paper, namely that WDGRL is substantially more robust than the baselines under growing feature shift in this semi-synthetic setting. Because the exact values are not identical and I report a single replication without confidence intervals, I do not claim that the numerical results are reproduced exactly.

8. Robustness Experiments

The authors do not rely solely on the main θ-experiment. They investigate several alternative settings to check that the conclusions do not depend on one particular configuration.

Lower production capacity

The main setting uses capacity $K_t = 70$; the authors also test $K_t = 50$. Lower capacity means that more orders compete for the available production slots, which makes scheduling harder and amplifies the consequences of inaccurate predictions. The main conclusion remains: WDGRL outperforms the ordinary machine learning baselines under the tested shifts.

Higher tardiness cost

The main experiment uses equal per-time-unit costs, $c_{\text{early}} = c_{\text{tardy}} = 1$. The authors also test a setting in which tardiness is twice as expensive as earliness, $c_{\text{early}} = 1$ and $c_{\text{tardy}} = 2$, which reflects the common situation in which late delivery is more damaging than early completion. WDGRL again achieves the lowest cost at the larger shifts.

Different noise distribution

The authors replace the Gaussian noise with uniform noise of comparable variance. A model might perform well simply because the noise distribution of the experiment happens to suit it; changing the noise distribution tests whether the result depends on Gaussian tails. The results continue to favor WDGRL.

Different outcome-generating function

The authors replace the random forest generating function with a gradient boosting model. This is an important check because the generating function is itself estimated from data; if the results depended heavily on the particular function used to generate the labels, the main finding would be less convincing. WDGRL continues to outperform the baselines in this alternative experiment.

Comparison with my replication

Figures 9 and 10 show the main result together with the lower-capacity and higher-cost-ratio settings, for the paper and for my replication, respectively. The noise-distribution and generating-function checks are reported in the paper and are not shown here. In both figures, the cost of the baselines increases steeply with θ while that of WDGRL grows slowly. My replication also reveals a limitation of the advantage at mild shift: in the higher-cost-ratio panel at $\theta = 1$, WDGRL has a higher cost than both baselines, whereas in the paper the three methods are roughly equal at this point. Together with the non-monotone WDGRL curves in my results (a drop at $\theta = 4$), this indicates that the run-to-run variability is comparable to the differences between methods when the shift is small, and that the advantage of WDGRL is reliable mainly for moderate and large shifts.

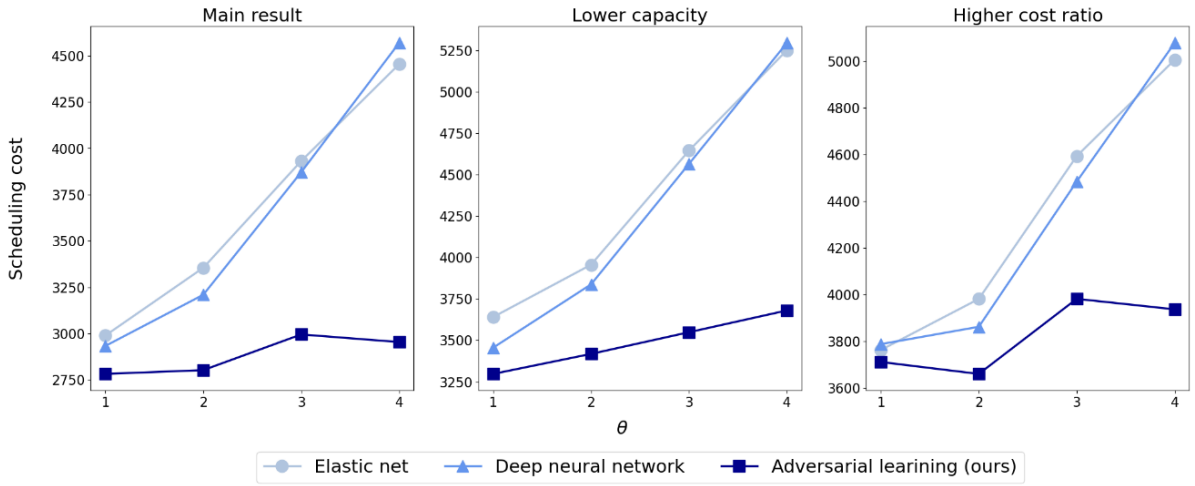


Figure 9: Scheduling cost reported by the authors for the main setting, lower capacity, and higher cost ratio

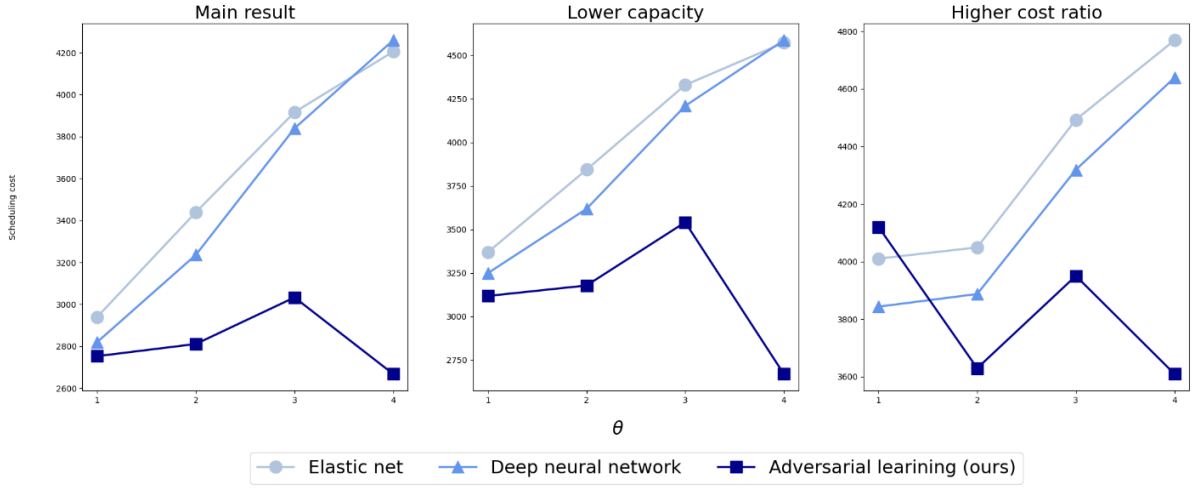


Figure 10: Scheduling cost from my replication for the main setting, lower capacity, and higher cost ratio

9. Limitations and Future Work

Limitations

- **Covariate shift only.** The method addresses changes in the feature distribution. If the physical production process itself changes, so that the relationship between features and throughput time is different, the alignment of representations will not correct the resulting error.
- **Similarity of old and new orders.** Although the two production periods do not overlap in time, the method implicitly assumes that old and new orders are similar enough in feature space to be aligned. The authors do not test this assumption formally.
- **Semi-synthetic design.** The main experiments use real Aker features but simulated throughput times. This permits controlled shifts but is not a pure field test. The Gaussian noise also produces negative throughput times (Figures 4 and 8), which are not physically meaningful.
- **Simplified scheduling.** Scheduling is limited to the 100 orders with the earliest due dates and a single capacity-constrained line, which is a simplification of a real job shop.
- **Single solver and decoupled design.** The optimization always uses the GLPK solver, and prediction and optimization remain separate stages; decision-aware learning approaches are not explored.

- **Baseline architecture.** The neural network is deliberately small and identical for the baseline and WDGRL. A larger or more heavily tuned baseline might narrow the gap.

Future work

Natural extensions include testing other optimization methods and solvers; evaluating larger neural network baselines and alternative domain adaptation methods; repeating the experiments over many random seeds to quantify variability; and validating the approach on real post-shift outcome data once these become available, which would provide a direct field test of the semi-synthetic findings.

10. Conclusion

The strongest contribution of the paper is its integration of domain adaptation with an operational scheduling decision. It shows how unlabeled data from a forthcoming production setting can be used during training to reduce the effect of feature-distribution shift, while preserving supervised prediction accuracy on historical data. Its key methodological insight is to combine the source prediction loss with a target representation alignment term: the Wasserstein critic estimates distributional differences in the latent space, the feature extractor reduces them, and the regressor preserves useful predictive information.

The numerical experiments show substantial improvements over the tested baselines, especially at larger simulated shifts, and the robustness checks strengthen the evidence under several alternative assumptions. My own replication supports this conclusion: although the numerical values differ moderately from the published ones, the ranking of the methods and the stability of WDGRL under increasing shift are reproduced.

The main qualification is that the results establish an advantage under the paper's semi-synthetic design and tested operational settings, not a universal guarantee for real-world production. The method relies on assumptions about the stability of the outcome relationship and the similarity of the source and target domains, and its practical value should ultimately be confirmed on real post-shift data.

References

Senoner, J., Kratzwald, B., Kuzmanovic, M., Netland, T., & Feuerriegel, S. (2023). Addressing distributional shifts in operations management: The case of order fulfillment in customized production. *Production and Operations Management*, 32(10).

Shen, J., Qu, Y., Zhang, W., & Yu, Y. (2018). Wasserstein distance guided representation learning for domain adaptation. In *Proceedings of the AAAI Conference on Artificial Intelligence*.